

EXPERIMENTS 11-20

Name: Naresh M

Reg no: 192424381

Course code: DSA0503

Subject : Query Processing for Data Science

EXPERIMENT – 11

Aim

To create a DataFrame of **10 rows and 4 columns** with random values and write a Pandas program to highlight the **negative numbers in red** and **positive numbers in black**.

Procedure

Step 1: Import the Pandas and NumPy libraries.

Step 2: Create a DataFrame with 10 rows and 4 columns.

Step 3: Generate random numbers between -20 and 20.

Step 4: Define a function to color the values.

Step 5: Apply the style using `applymap()`.

Step 6: Display the styled DataFrame.

Code:

```
import pandas as pd
import numpy as np
np.random.seed(10)
df = pd.DataFrame(
    np.random.randint(-20, 21, size=(10,4)),
    columns=['A','B','C','D']
)
def color_value(val):
    if val < 0:
        return 'color:red'
    else:
        return 'color:black'
print("Original DataFrame")
print(df)
```

```
df.style.applymap(color_value)
```

Output:

```
... Original DataFrame
```

	A	B	C	D
0	-11	16	-5	-20
1	8	5	9	9
2	-12	-11	-20	20
3	16	-4	16	-9
4	4	13	-12	16
5	-6	-7	-15	-7
6	5	-7	8	2
7	10	10	5	-8
8	-19	11	16	7
9	-2	9	-7	2

```
/tmp/ipykernel_2525/992  
df.style.applymap(col
```

	A	B	C	D
0	-11	16	-5	-20
1	8	5	9	9
2	-12	-11	-20	20
3	16	-4	16	-9
4	4	13	-12	16
5	-6	-7	-15	-7
6	5	-7	8	2
7	10	10	5	-8
8	-19	11	16	7
9	-2	9	-7	2

Result

Thus, the Pandas program to create a DataFrame with random values and **highlight negative numbers in red** and **positive numbers in black** was executed successfully.

EXPERIMENT – 12

Aim

To create a DataFrame of **10 rows and 4 columns** with random values and write a Pandas program to set the **DataFrame background color to black** and **font color to yellow**

Procedure

Step 1: Import the Pandas and NumPy libraries.

Step 2: Create a DataFrame with **10 rows and 4 columns** containing random integers.

Step 3: Display the original DataFrame.

Step 4: Apply a style to set the background color to **black** and the font color to **yellow**.

Step 5: Display the styled DataFrame.

Code:

```
import pandas as pd
import numpy as np
np.random.seed(10)
df = pd.DataFrame(
    np.random.randint(-20, 21, (10,4)),
    columns=["A","B","C","D"]
)
print("Original DataFrame")
print(df)
styled_df = df.style.set_properties(
    **{
        'background-color': 'black',
        'color': 'yellow'
    }
)
styled_df
```

Output:

Original DataFrame						A	B	C	D
...					0	-11	16	-5	-20
	A	B	C	D	1	8	5	9	9
0	-11	16	-5	-20	2	-12	-11	-20	20
1	8	5	9	9	3	16	-4	16	-9
2	-12	-11	-20	20	4	4	13	-12	16
3	16	-4	16	-9	5	-6	-7	-15	-7
4	4	13	-12	16	6	5	-7	8	2
5	-6	-7	-15	-7	7	10	10	5	-8
6	5	-7	8	2	8	-19	11	16	7
7	10	10	5	-8	9	-2	9	-7	2
8	-19	11	16	7					
9	-2	9	-7	2					

Result

Thus, the Pandas program to create a DataFrame with random values and **set the background color to black and font color to yellow** was executed successfully, and the styled DataFrame was displayed.

Experiment 13

Aim

To write a Pandas program to detect the missing values in a given DataFrame and display the result as True or False.

Procedure

Step 1: Import the Pandas and NumPy libraries.

Step 2: Create a DataFrame containing some missing (NaN) values.

Step 3: Display the original DataFrame.

Step 4: Use the isnull() function to detect missing values.

Step 5: Display the result as True (missing value) and False (non-missing value).

Code:

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    "A": [10, 15, 25, np.nan, 80],
    "B": [20, np.nan, 30, 50, 90],
    "C": [np.nan, 35, 40, 60, 100],
    "D": [40, 45, np.nan, 70, 110]
})
print("Original DataFrame")
print(df)
print("\nMissing Values (True/False)")
print(df.isnull())
```

Output:

Original DataFrame				
	A	B	C	D
0	10.0	20.0	NaN	40.0
1	15.0	NaN	35.0	45.0
2	25.0	30.0	40.0	NaN
3	NaN	50.0	60.0	70.0
4	80.0	90.0	100.0	110.0

Missing Values (True/False)				
	A	B	C	D
0	False	False	True	False
1	False	True	False	False
2	False	False	False	True
3	True	False	False	False
4	False	False	False	False

Result

Thus, the Pandas program to **detect missing values** in the given DataFrame was executed successfully, and the missing values were displayed as **True** and non-missing values as **False**.

EXPERIMENT – 14

Aim

To write a Pandas program to replace the missing values (NaN) in a DataFrame with a specified value.

Procedure

Step 1: Import the Pandas and NumPy libraries.

Step 2: Create a DataFrame containing some missing (NaN) values.

Step 3: Display the original DataFrame.

Step 4: Replace all missing values using the fillna() function.

Step 5: Display the updated DataFrame.

Code:

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    "A": [10, 15, 25, np.nan, 80],
    "B": [20, np.nan, 30, 50, 90],
    "C": [np.nan, 35, 40, 60, 100],
    "D": [40, 45, np.nan, 70, 110]
})
print("Original DataFrame")
print(df)
```

```
new_df = df.fillna(0)

print("\nDataFrame after replacing missing values")

print(new_df)
```

Output:

Original DataFrame				
	A	B	C	D
0	10.0	20.0	NaN	40.0
1	15.0	NaN	35.0	45.0
2	25.0	30.0	40.0	NaN
3	NaN	50.0	60.0	70.0
4	80.0	90.0	100.0	110.0

Missing values (True/False)				
	A	B	C	D
0	False	False	True	False
1	False	True	False	False
2	False	False	False	True
3	True	False	False	False
4	False	False	False	False

Result

Thus, the Pandas program to replace missing values (NaN) with a specified value was executed successfully, and the updated DataFrame was displayed.

EXPERIMENT – 15

Aim

To write a Pandas program to keep only the rows containing at least two NaN (missing) values in a given DataFrame.

Procedure

Step 1: Import the Pandas and NumPy libraries.

Step 2: Create a DataFrame containing some missing (NaN) values.

Step 3: Display the original DataFrame.

Step 4: Count the missing values in each row using `isnull().sum(axis=1)`.

Step 5: Select only those rows where the number of missing values is greater than or equal to 2.

Step 6: Display the filtered DataFrame.

Code:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "A": [10, np.nan, 25, np.nan, 80],
```

```

"B": [20, np.nan, 30, 50, 90],
"C": [np.nan, 35, 40, np.nan, 100],
"D": [40, 45, np.nan, 70, 110]
})

print("Original DataFrame")

print(df)

result = df[df.isnull().sum(axis=1) >= 2]

print("\nRows with at least 2 NaN values")

print(result)

```

Output:

Original DataFrame				
	A	B	C	D
0	10.0	20.0	NaN	40.0
1	NaN	NaN	35.0	45.0
2	25.0	30.0	40.0	NaN
3	NaN	50.0	NaN	70.0
4	80.0	90.0	100.0	110.0

Rows with at least 2 NaN values				
	A	B	C	D
1	NaN	NaN	35.0	45.0
3	NaN	50.0	NaN	70.0

Result

Thus, the Pandas program to **keep the rows with at least two NaN values** in the given DataFrame was executed successfully, and the required rows were displayed.

EXPERIMENT – 16

Aim

To write a Pandas program to split the given DataFrame into groups based on the school_code column and check the type of the GroupBy object

Procedure

Step 1: Import the Pandas library.

Step 2: Create a DataFrame with the given school data.

Step 3: Display the original DataFrame.

Step 4: Use the groupby() function to group the data by school_code.

Step 5: Display each group.

Step 6: Display the type of the GroupBy object.

Code:

```
import pandas as pd

df = pd.DataFrame({
    "school_code": ["S1","S2","S3","S4","S5","S6"],
    "class": ["V","V","VI","VI","V","VI"],
    "name": [
        "Alberto Franco",
        "Gino Mcneill",
        "Ryan Parkes",
        "Eesha Hinton",
        "Gino Mcneill",
        "David Parkes"
    ],
    "date_of_birth": [
        "15/05/2002",
        "17/05/2002",
        "16/09/1999",
        "25/09/1998",
        "11/05/2002",
        "15/09/1997"
    ],
    "age": [12,12,13,13,14,12],
    "height": [173,192,186,167,151,159],
    "weight": [35,32,33,30,31,32],
    "address": [
        "street1",
        "street2",
        "street3",
        "street1",
        "street2",
        "street4"
    ]
})
```



```

})

print("Original DataFrame")

print(df)

group = df.groupby("school_code")

for name, data in group:

    print("\nSchool Code:", name)

    print(data)

print("\nType of GroupBy Object:")

print(type(group))

```

Output:

```

Original DataFrame
...  school_code  class  name  date_of_birth  age  height  weight  \
0      S1      V  Alberto Franco  15/05/2002  12    173    35
1      S2      V   Gino Mcneill  17/05/2002  12    192    32
2      S3     VI   Ryan Parkes  16/09/1999  13    186    33
3      S4     VI   Eesha Hinton  25/09/1998  13    167    30
4      S5      V   Gino Mcneill  11/05/2002  14    151    31
5      S6     VI   David Parkes  15/09/1997  12    159    32

address
0  street1
1  street2
2  street3
3  street1
4  street2
5  street4

```

```

School Code: S1
  school_code  class  name  date_of_birth  age  height  weight  \
0      S1      V  Alberto Franco  15/05/2002  12    173    35

address
0  street1

School Code: S2
  school_code  class  name  date_of_birth  age  height  weight  address
1      S2      V   Gino Mcneill  17/05/2002  12    192    32  street2

School Code: S3
  school_code  class  name  date_of_birth  age  height  weight  address
2      S3     VI   Ryan Parkes  16/09/1999  13    186    33  street3

School Code: S4
  school_code  class  name  date_of_birth  age  height  weight  address
3      S4     VI   Eesha Hinton  25/09/1998  13    167    30  street1

School Code: S5
  school_code  class  name  date_of_birth  age  height  weight  address
4      S5      V   Gino Mcneill  11/05/2002  14    151    31  street2

School Code: S6
  school_code  class  name  date_of_birth  age  height  weight  address
5      S6     VI   David Parkes  15/09/1997  12    159    32  street4

Type of GroupBy Object:
<class 'pandas.core.groupby.generic.DataFrameGroupBy'>

```

Result

Thus, the Pandas program to split the DataFrame into groups based on `school_code` and check the type of the GroupBy object was executed successfully, and the required grouped output and object type were displayed.

EXPERIMENT – 17

Aim

To write a Pandas program to **split the DataFrame by `school_code`** and find the **mean, minimum, and maximum values of `age`** for each school.

Procedure

Step 1: Import the Pandas library.

Step 2: Create a DataFrame with the given school data.

Step 3: Display the original DataFrame.

Step 4: Group the DataFrame by **`school_code`** using `groupby()`.

Step 5: Calculate the **mean, minimum, and maximum** values of the **`age`** column.

Step 6: Display the result.

Code:

```
import pandas as pd

df = pd.DataFrame({
    "school_code": ["S1", "S2", "S3", "S4", "S5", "S6"],
    "class": ["V", "V", "VI", "VI", "V", "VI"],
    "name": [
        "Alberto Franco",
        "Gino Mcneill",
        "Ryan Parkes",
        "Eesha Hinton",
        "Gino Mcneill",
        "David Parkes"
    ],
    "date_of_birth": [
        "15/05/2002",
        "17/05/2002",
        "16/09/1999",
```

```

    "25/09/1998",
    "11/05/2002",
    "15/09/1997"
],
"age": [12,12,13,13,14,12],
"height": [173,192,186,167,151,159],
"weight": [35,32,33,30,31,32],
"address": [
    "street1",
    "street2",
    "street3",
    "street1",
    "street2",
    "street4"
]
})

print("Original DataFrame")

print(df)

result = df.groupby("school_code")["age"].agg(["mean","min","max"])

print("\nMean, Minimum and Maximum Age for Each School")

print(result)

```

Output:

```

Original DataFrame
  school_code class      name date_of_birth  age  height  weight \
0         S1     V  Alberto Franco  15/05/2002   12    173     35
1         S2     V   Gino Mcneill  17/05/2002   12    192     32
2         S3    VI    Ryan Parkes  16/09/1999   13    186     33
3         S4    VI   Eesha Hinton  25/09/1998   13    167     30
4         S5     V   Gino Mcneill  11/05/2002   14    151     31
5         S6    VI   David Parkes  15/09/1997   12    159     32

  address
0 street1
1 street2
2 street3
3 street1
4 street2
5 street4

Mean, Minimum and Maximum Age for Each School
      mean  min  max
school_code
S1      12.0   12   12
S2      12.0   12   12
S3      13.0   13   13
S4      13.0   13   13
S5      14.0   14   14
S6      12.0   12   12

```

Result

Thus, the Pandas program to split the DataFrame by school_code and calculate the mean, minimum, and maximum values of age for each school was executed successfully, and the required output was displayed.

EXPERIMENT – 17

Aim

To write a Pandas program to **split the DataFrame by school_code** and calculate the **mean age** of students in each school.

Procedure

Step 1: Import the Pandas library.

Step 2: Create the DataFrame with school details.

Step 3: Display the original DataFrame.

Step 4: Group the DataFrame using the school_code column.

Step 5: Calculate the **mean** of the age column.

Step 6: Display the result.

Code:

```
import pandas as pd

df = pd.DataFrame({
    "school_code": ["S1","S2","S3","S4","S5","S6"],
    "class": ["V","V","VI","VI","V","VI"],
    "name": [
        "Alberto Franco",
        "Gino Mcneill",
        "Ryan Parkes",
        "Eesha Hinton",
        "Gino Mcneill",
        "David Parkes"
    ],
    "date_of_birth": [
        "15/05/2002",
        "17/05/2002",
        "16/09/1999",
        "25/09/1998",
```

```

    "11/05/2002",
    "15/09/1997"
],
"age": [12,12,13,13,14,12],
"height": [173,192,186,167,151,159],
"weight": [35,32,33,30,31,32],
"address": [
    "street1",
    "street2",
    "street3",
    "street1",
    "street2",
    "street4"
]
})

print("Original DataFrame")

print(df)

result = df.groupby("school_code")["age"].mean()

print("\nMean Age for Each School")

print(result)

```

Output:

```

... Original DataFrame
  school_code class      name date_of_birth  age  height  weight \
0          S1    V  Alberto Franco   15/05/2002   12    173    35
1          S2    V    Gino Mcneill   17/05/2002   12    192    32
2          S3   VI    Ryan Parkes   16/09/1999   13    186    33
3          S4   VI    Eesha Hinton   25/09/1998   13    167    30
4          S5    V    Gino Mcneill   11/05/2002   14    151    31
5          S6   VI    David Parkes   15/09/1997   12    159    32

  address
0 street1
1 street2
2 street3
3 street1
4 street2
5 street4

Mean Age for Each School
school_code
S1      12.0
S2      12.0
S3      13.0
S4      13.0
S5      14.0
S6      12.0
Name: age, dtype: float64

```

Result

Thus, the Pandas program to split the DataFrame by school_code and calculate the mean age was executed successfully, and the required output was displayed.

EXPERIMENT – 18

Aim

To write a Pandas program to **split the given DataFrame into groups based on school_code and class.**

Procedure

Step 1: Import the Pandas library.

Step 2: Create the DataFrame with school details.

Step 3: Display the original DataFrame.

Step 4: Group the DataFrame using **school_code** and **class**.

Step 5: Display each group separately.

Code:

```
import pandas as pd

df = pd.DataFrame({
    "school_code": ["S1","S2","S3","S4","S5","S6"],
    "class": ["V","V","VI","VI","V","VI"],
    "name": [
        "Alberto Franco",
        "Gino Mcneill",
        "Ryan Parkes",
        "Eesha Hinton",
        "Gino Mcneill",
        "David Parkes"
    ],
    "date_of_birth": [
        "15/05/2002",
        "17/05/2002",
        "16/09/1999",
        "25/09/1998",
        "11/05/2002",
        "14/08/2001"
    ]
})
```

```

        "15/09/1997"

    ],
    "age": [12,12,13,13,14,12],
    "height": [173,192,186,167,151,159],
    "weight": [35,32,33,30,31,32],
    "address": [
        "street1",
        "street2",
        "street3",
        "street1",
        "street2",
        "street4"
    ]
})

print("Original DataFrame")

print(df)

group = df.groupby(["school_code", "class"])

for name, data in group:

    print("\nGroup:", name)

    print(data)

```

Result:

```

Original DataFrame
  school_code class      name date_of_birth  age  height  weight \
0          S1     V  Alberto Franco  15/05/2002   12    173     35
1          S2     V   Gino Mcneill  17/05/2002   12    192     32
2          S3    VI   Ryan Parkes  16/09/1999   13    186     33
3          S4    VI  Eesha Hinton  25/09/1998   13    167     30
4          S5     V   Gino Mcneill  11/05/2002   14    151     31
5          S6    VI   David Parkes  15/09/1997   12    159     32

  address
0  street1
1  street2
2  street3
3  street1
4  street2
5  street4

```

```

Group: ('S1', 'V')
  school_code class      name date_of_birth  age  height  weight \
0          S1    V  Alberto Franco   15/05/2002   12    173    35

  address
0  street1

Group: ('S2', 'V')
  school_code class      name date_of_birth  age  height  weight  address
1          S2    V   Gino Mcneill   17/05/2002   12    192    32  street2

Group: ('S3', 'VI')
  school_code class      name date_of_birth  age  height  weight  address
2          S3   VI   Ryan Parkes   16/09/1999   13    186    33  street3

Group: ('S4', 'VI')
  school_code class      name date_of_birth  age  height  weight  address
3          S4   VI   Eesha Hinton   25/09/1998   13    167    30  street1

Group: ('S5', 'V')
  school_code class      name date_of_birth  age  height  weight  address
4          S5    V   Gino Mcneill   11/05/2002   14    151    31  street2

Group: ('S6', 'VI')
  school_code class      name date_of_birth  age  height  weight  address
5          S6   VI   David Parkes   15/09/1997   12    159    32  street4

```

Result

Thus, the Pandas program to split the DataFrame into groups based on school_code and class was executed successfully, and the grouped records were displayed.

EXPERIMENT- 19

Aim

To write a Pandas program to display the dimensions (shape) of the World Alcohol Consumption dataset and extract the column names from the dataset.

Procedure

Step 1: Import the Pandas library.

Step 2: Read the world_alcohol.csv dataset using pd.read_csv().

Step 3: Display the complete DataFrame.

Step 4: Find the dimensions of the DataFrame using the shape attribute.

Step 5: Extract the column names using the columns attribute.

Step 6: Display the shape and column names.

Code:

```

import pandas as pd

df = pd.read_csv("world_alcohol.csv")

```



```

print("World Alcohol Consumption Dataset")

print(df)

print("\nDimensions (Rows, Columns):")

print(df.shape)

print("\nColumn Names:")

print(df.columns)

```

Output:

```

===== RESTART: C:/Users/nares/Videos/dsa0503/experiments/exp 19.py =====
World Alcohol Consumption Dataset
   Year  WHO Region  Country Beverage Types  Display Value
0  1986  Western Pacific  Viet Nam      Wine           0.00
1  1986      Americas  Uruguay      Other           0.50
2  1985      Africa  Cote d'Ivoire      Wine           1.62
3  1986      Americas  Colombia      Beer            4.27
4  1987      Americas  Saint Kitts and Nevis  Beer            1.98

Dimensions (Rows, Columns):
(5, 5)

Column Names:
Index(['Year', 'WHO Region', 'Country', 'Beverage Types', 'Display Value'], dtype='str')

```

Result

Thus, the Pandas program to display the dimensions (shape) of the World Alcohol Consumption dataset and extract the column names was executed successfully.

EXPERIMENT - 20

Aim

To write a Pandas program to find the index of a given substring in a DataFrame column.

Procedure

Step 1: Import the Pandas library.

Step 2: Create a DataFrame with a column containing string values.

Step 3: Display the original DataFrame.

Step 4: Use the `str.find()` function to locate the position (index) of the required substring.

Step 5: Store the result in a new column.

Step 6: Display the updated DataFrame.

Code:

```

import pandas as pd

df = pd.DataFrame({
    "Name": [

```

```

    "Albert Franco",
    "Gino Mcneill",
    "Ryan Parkes",
    "Eesha Hinton",
    "David Parkes"
]
})
print("Original DataFrame")
print(df)
df["Substring_Index"] = df["Name"].str.find("Park")
print("\nDataFrame after finding substring index")
print(df)

```

Output:

```

Original DataFrame
      Name
0  Albert Franco
1   Gino Mcneill
2    Ryan Parkes
3   Eesha Hinton
4   David Parkes

DataFrame after finding substring index
      Name  Substring_Index
0  Albert Franco         -1
1   Gino Mcneill         -1
2    Ryan Parkes          5
3   Eesha Hinton         -1
4   David Parkes          6

```

Result

Thus, the Pandas program to find the index of a given substring in a DataFrame column was executed successfully, and the index positions of the substring were displayed.